# LLD Notes — Phase 3, Part 4

Decorator · Adapter · Command · Jumbled round 2 (PuneGo) | Stories, your doubts, your mistakes, interview answers

> **Colour key. Blue = a doubt you asked · Red = a mistake you made · Green = rule to remember · Amber = how to say it in an interview.**

**Contents**


**15. Decorator** — Bhau's chai stall · wrapping · order matters · exercise 24 (Katta) · compress then encrypt · failure rules

**16. Adapter** — the travel plug · what an adapter translates · exercise 25 (Dabba) · keep adapters thin · the lat/lng swap

**17. Command** — shout vs slip · the four powers · two kinds of undo · exercise 26 (Ghar) · redo · saving commands

**18. Jumbled round 2** (PuneGo) — eleven requirements · how the patterns fit together · `bookTicket()`

**19. The hands-on build** — four stages, five patterns woven into one service

**20.** Cheat sheet for all nine patterns · new rules · mistake tracker · recall questions


## 15. Decorator

### The story: Bhau's chai stall

Bhau sells two bases (cutting chai Rs 15, masala chai Rs 20) and four add-ons (adrak +5, elaichi +5, malai +10, kulhad +10). His nephew Sahil builds one class per combination: `MasalaChaiWithAdrakAndElaichi`, and so on.

| Question | Answer |
|---|---|
| 2 bases, 4 add-ons: how many classes? | **32** (2 × 2 × 2 × 2 × 2) |
| Add one more add-on (honey)? | **64**. Every new add-on doubles the count. |
| A customer wants *double* adrak? | **Impossible.** Inheritance can't say "the same thing twice." |

> **Your instinct: "one chai class with addAdrak(), addElaichi(), like a Builder."** The key idea in it was right: **start with a base and add layers**. The catch: every new add-on means editing that one class. Decorator keeps your layering idea, but makes **each add-on its own small object**.

### The fix: wrap it

```
interface Chai { int price(); }

class MasalaChai implements Chai {
    public int price() { return 20; }
}

class Adrak implements Chai {
    private final Chai inner;                      // whatever is inside me
    Adrak(Chai inner) { this.inner = inner; }
    public int price() { return inner.price() + 5; }   // what's inside, plus me
}

Chai order  = new Kulhad(new Elaichi(new Adrak(new MasalaChai())));   // 40
Chai double = new Adrak(new Adrak(new MasalaChai()));                 // 30, double adrak works
```

Honey next month is one new tiny class. **6 classes instead of 64.**

```
Kulhad.price()
  → asks Elaichi
      → asks Adrak
          → asks MasalaChai  → 20
      ← Adrak:   20 + 5  = 25
  ← Elaichi:     25 + 5  = 30
← Kulhad:        30 + 10 = 40
```

> **The trick that makes it work:** `class Adrak implements Chai`. An add-on **is also a Chai**. That's why it can be wrapped again, and why anything that accepts a Chai accepts one wrapped five times.
> **Decorator = same interface as the thing it wraps, adds a bit on top.**

### You had already used it

```
new BufferedReader(new InputStreamReader(new FileInputStream("data.txt")));
List<String> safe     = Collections.unmodifiableList(names);   // same List, add() throws
List<String> threaded = Collections.synchronizedList(names);   // same List, every call locked
```

### Decorator on top of Strategy: the parking lot weekend rule

You designed this yourself on the first try: a "weekend" class that implements the pricing strategy, takes another strategy, and multiplies its price by 1.5.

```
PricingStrategy car =
    new MemberDiscount(              // 10% off
        new FestivalSurcharge(       // +20
            new WeekendSurcharge(    // x1.5
                new CarPricing())));
```

**Order matters.** Base 80, a member, on a Diwali Saturday:
**Member( Festival( Weekend( Car ) ) )** → 80 × 1.5 = 120 → +20 = 140 → −10% = **126**
**Festival( Weekend( Member( Car ) ) )** → 80 −10% = 72 → × 1.5 = 108 → +20 = **128**
Same four rules, different answer. The first is better for the customer; the second earns the mall more. **You said it correctly: Finance decides, not engineering.**

**Say it:** "Decorators don't commute, so the wrapping order is a business decision. I'd keep the wiring in one place, a factory, so the policy is visible and Finance can review it."

| | Strategy | Decorator |
|---|---|---|
| What it does | **replaces** the algorithm | **adds around** it |
| How many at once | you pick one | you stack many |
| Example | car pricing *or* truck pricing | weekend *and* festival *and* member |

**Downsides, to sound balanced:** many tiny objects; debugging means walking through layers; a wrapped object is not `==` the original and `instanceof CarPricing` fails on it; you can't remove a middle layer, you rebuild the stack.

## Exercise 24 — Katta (a college chat app)

| Req | What it said | Answer | You |
|---|---|---|---|
| R1 | Admins can turn on a profanity filter | Decorator | ✓ |
| R2 | Secret chats are encrypted | Decorator | ✓ |
| R3 | Messages over 1 KB are compressed | Decorator | ✓ |
| R4 | **Every** message gets a timestamp | **Nothing**, a field on the message | ✓ **sharp** |
| R5 | Official channel adds a "Verified" footer | Decorator | ½ (said Observer, then self-corrected) |
| R6 | After sending: last seen, push, analytics | Observer | ✓ |
| R7 | WiFi, mobile data, or SMS fallback | Strategy | ✓ |
| R8 | Exactly one emoji style, forever | Nothing | ✓ |

**Two rules from Q1**
**If it applies to everything, every time, put it in the base.** Decorators are for the things you switch on and off. (R4)
**Does it change the thing passing through?** Yes → Decorator. No, it reacts afterwards → Observer. (R5 vs R6)

### Q2 — compress first or encrypt first?

**You said: encrypt, then compress.** Your reason was "compressed data isn't the actual data, so it can't be encrypted." Encryption doesn't care what the bytes mean; it can lock any bytes. The order is the other way round, and here is the experiment I ran on a 2 KB message with real AES:

```
original              2040 bytes
compress → encrypt     106 bytes     correct
encrypt → compress    2067 bytes     bigger than the original
```

**Why:** compression works by finding repeats ("hahahaha" → "ha × 4"). Encryption's job is to destroy repeats so the output looks random. Encrypt first and there is nothing left for the compressor to find.

```
MessageProcessor pipeline =
    new Encrypt(                // 3. lock it up last
        new Compress(           // 2. shrink it while it still has patterns
            new ProfanityFilter( // 1. read the words while they're still words
                new PlainText())));
```

**Rules:** **Compress, then encrypt.** Encrypted data looks random, and random data doesn't compress.
**A layer that reads the content must run while the content is still in the form it understands.** That puts the filter first. ✓ **You had the filter first.**

### Q4 — the receiving phone

✓ **You got it, and corrected yourself on the filter.** Unwrapping goes in reverse: decrypt, then decompress. Last on, first off, like a gift.

| Layer | Can it be reversed? |
|---|---|
| Compress | Yes, exactly |
| Encrypt | Yes, with the key |
| Profanity filter | **No, and it shouldn't be.** The friend is meant to see the stars. |

**Extra credit:** only messages over 1 KB are compressed, so the receiver can't blindly decompress. The sender attaches a label like `{compressed: true, encrypted: true}`. HTTP does the same with `Content-Encoding: gzip`.

### Q5 — double encryption

✓ **Right conclusion: no benefit.** Small correction: the receiver *can* decrypt twice; it just buys nothing. The design lesson: decorators allow any stacking, including nonsense. So build the stack in one place that only produces valid pipelines:

```
class MessagePipelineFactory {
    MessageProcessor build(boolean profanityOn, boolean official, boolean secret, int length) {
        MessageProcessor p = new PlainText();
        if (profanityOn)   p = new ProfanityFilter(p);
        if (official)      p = new VerifiedFooter(p);
        if (length > 1024) p = new Compress(p);
        if (secret)        p = new Encrypt(p);       // always last, always once
        return p;
    }
}
```

### Your Part B

**✓ The decorator shape was right five times out of five, and two old mistakes were gone: every override was `public`, every call had its receiver.**

**Bug 1 — the footer threw away the inner result.**

```
String actualMessage = processor.process(message);
String addFooter = message + "Verified by SPPU";      // message, not actualMessage
```

Input "exam postponed you idiot" came out as "exam postponed you idiotVerified by SPPU". The filter ran and was discarded. **This is the classic decorator bug, and it compiles.** Fix that makes it impossible: don't create the variable.

```
return processor.process(message) + " Verified by SPPU";
```

**Bug 2:** `new StringBuilder(input)`, and no variable called `input` exists.
**Bug 3:** the factory put **every** layer on **every** message, and returned a String. Layers are optional; and **a factory returns, it doesn't act**.
**Also:** six `public class` in one file; `main` printed "Hello World".

### Part C

**Q1 — add "translate to Marathi"** ✓ One new class, plus one line and one parameter in the factory. Translate goes **after** the filter: the filter's word list is English, so translating first lets the bad word walk past in a language the list doesn't know.
**Five boolean flags is a smell.** `build(true, false, true, false, 300)` is unreadable; pass a small `ChatSettings` object.

**Q2 — a layer fails. Right decision, opposite mechanism.** You said: don't send if Encrypt fails ✓, so add try/catch in every layer ✗. A catch block lets the code **carry on**, which sends the secret message as plain text.
**try/catch is for when you want to continue. If you want to stop, let it throw.** Stopping is the default.

| Layer fails | The skipped message is… | So |
|---|---|---|
| Encrypt | a secret sent in the open | **stop** (fail closed) |
| Profanity filter | unfiltered, where the admin asked for filtering | stop, most likely |
| Compress | the same message, a bit bigger | **send anyway** (fail open) |
| Verified footer | correct, missing its badge | ask the product owner |

The test: **if this layer is skipped, is the message still safe and correct to send?** It is the same question as "listener or required step?"

**Q3 — "one class with four ifs would do."** You defended the factory, but the challenge was about the five decorator classes. Both versions have four ifs: theirs wrap the *logic*, ours only *assemble*.
**Script:** "For four small fixed steps, I'd agree, a single class is simpler. I'd switch to decorators when the steps grow into real logic, when new ones keep being added, or when other features need the same steps in different combinations."
**The habit:** when you hear "over-engineered", start with "*You'd be right if…*"

# 16. Adapter

## The story: the travel plug

Your cousin's US laptop charger has flat pins. The wall socket in Pune has round holes. You can't rebuild the wall and you can't cut open the charger. You use an adapter.

In code: Bhau's app calls `PaymentGateway.pay(int amountInRupees)` in 40 places. The new provider's SDK, a jar you can't edit, offers `makeTransaction(long amountInPaise, String currencyCode, String merchantId)`.

> **You designed it yourself:** "create a class which implements the existing payment gateway, inject the SDK, and call makeTransaction inside pay."

```
class QuickPeAdapter implements PaymentGateway {      // looks like what the app expects
    private final QuickPeSdk sdk;                      // holds the thing that doesn't fit
    private final String merchantId;

    QuickPeAdapter(QuickPeSdk sdk, String merchantId) { this.sdk = sdk; this.merchantId = merchantId; }

    @Override
    public void pay(int amountInRupees) {
        sdk.makeTransaction(amountInRupees * 100L, "INR", merchantId);   // translate
    }
}
```

| The app says | The vendor wants | The adapter |
|---|---|---|
| `pay` | `makeTransaction` | changes the **name** |
| rupees | paise | changes the **units** |
| just an amount | currency and merchant id too | **fills in** what's missing |

The 40 call sites: untouched. The SDK: untouched. A second vendor next year is one more small class. **Why the units line matters:** in 1999 NASA lost a Mars orbiter because one team sent pound-force and the other expected newtons. An adapter is the one place where the conversion lives.

> **On naming principles:** you listed OCP, Liskov and Interface Segregation. Only OCP applied. Name only the principles you can explain in one sentence each.

## Decorator vs Adapter

```
new BufferedReader(new InputStreamReader(new FileInputStream("data.txt")));
//  Decorator: Reader → Reader      Adapter: InputStream (bytes) → Reader (characters)
```

| | Decorator | Adapter |
|---|---|---|
| Wraps a… | `Chai` | `QuickPeSdk` |
| And is a… | `Chai` (same shape) | `PaymentGateway` (**different** shape) |
| Adds behaviour? | **Yes**, that's its job | **No**, it only translates |
| Stackable? | Yes | Usually just one |
| Why it exists | you want *more* | two things *don't fit* |

> **One line:** a decorator keeps the shape and changes the behaviour. An adapter keeps the behaviour and changes the shape.

## Exercise 25 — Dabba (tiffin delivery)

| Req | What it said | Answer | You |
|---|---|---|---|
| R1 | MapMyRide SDK doesn't match `LocationProvider` | Adapter | ✓ |
| R2 | TextKaro SMS vendor, needs `+91` | Adapter | ✓ |
| R3 | Log and retry every SMS, **for any vendor** | **Decorator** | **✗ put it inside the adapter** |
| R4 | Our **own** `OldBillingService.calc()` should be `calculateBill()` | **Nothing: rename it** | **✗ built a wrapper** |
| R5 | Customer chooses between three payment providers | Adapter × 3, then Strategy + Factory | ½ |

> **Keep adapters thin (R3).** An adapter only translates. The moment you want to *add* behaviour, that's a different class.
>
> ```
> Notifier notifier = new RetryingNotifier(new TextKaroAdapter(sdk));
> //                  decorator outside       adapter inside
> ```
>
> Swap the vendor and the logging and retry come along for free.

> **Adapter is for code you can't change (R4).** A vendor's jar, another team's service, a legacy system nobody dares touch. **If you own it, change it.** In your own house you'd change the socket, not buy a travel adapter.

### Q2 — what the R1 adapter must translate

| # | App expects | SDK gives | Found? |
|---|---|---|---|
| 1 | returns a `Location` | returns a `double[]` | ✓ |
| 2 | no checked exception | `throws MmrException` | ✓ |
| 3 | `String riderId` | `int riderCode` | **missed** |
| 4 | `Location(latitude, longitude)` | `{longitude, latitude}` | **missed** |

**#4 — the famous bug.** Both values are `double`, so `new Location(r[0], r[1])` compiles with no warning. Pune is at latitude 18.5, longitude 73.9. Swapped, the rider appears in the Arctic sea north of Norway.
**When two values have the same type, the compiler can't tell them apart.** Read the vendor's docs for order and units, and comment that line.

```
public Location current(String riderId) {
    try {
        int code = Integer.parseInt(riderId);                 // #3  String → int
        double[] lngLat = sdk.getLngLat(code);                // #1  their type
        return new Location(lngLat[1], lngLat[0]);            // #4  SDK is {lng, lat}; we need (lat, lng)
    } catch (MmrException e) {
        throw new LocationUnavailableException(riderId, e);   // #2  their error → our error
    }
}
```

**An adapter translates names, types, order, units and errors.** Everything vendor-specific stops at this class; the 40 callers never import a vendor exception.

**Q4 — the vendor changes ✓ (you self-corrected to include the wiring line).** New: one adapter. Changed: one wiring line. Deleted: the old adapter. Untouched: the interface, every caller, and the retry decorator.
**Say it:** "Swapping a vendor costs one new adapter and one line of wiring. Cross-cutting behaviour like retry lives in a decorator, so it carries over."

### Your Part B

✓ **You got the lat/lng order right, used `parseInt`, and wired decorator-outside-adapter correctly.**

1. `return LocationUnavailableException(riderId, e);` outside the catch. An exception is **thrown**, not returned (`throw new`); and `e` exists only inside the catch braces.
2. `if (phone.charAt(0) != '9' && phone.charAt(1) != 1)` skips the prefix for any number starting with 9, which is most of them. Use `phone.startsWith("+91") ? phone : "+91" + phone`.
3. `new TextKaroAdapter(new TkMessage())` passed a message where the SDK belongs. Naming the field `sdk`, not `msg`, prevents this.
4. `main`: missing `Scanner` import, a `return` in a void method, and keyboard input instead of the three fixed calls.

# 17. Command

## The story: shout vs slip

A busy Udupi restaurant with no order slips. The waiter shouts the order to the chef. At rush hour nobody knows who ordered what, or which order came first.

A shout in code is a normal method call. It happens now, and afterwards **nothing is left behind**:

```
chef.cook("masala dosa", 4);       // can't wait, can't be cancelled, leaves no record
```

A slip is **a thing you can hold**. In code, that's an object:

```
interface Command { void execute(); }

class CookOrder implements Command {
    private final Chef chef;        // who will do it
    private final String dish;      // what to do
    private final int table;        // the details
    public void execute() { chef.cook(dish, table); }     // the same call, just not yet
}

Queue<Command> rail = new LinkedList<>();
rail.add(new CookOrder(chef, "masala dosa", 4));     // waiter clips a slip
rail.poll().execute();                                // later: chef takes the next slip
```

**This didn't click at first, and you said so.** The second explanation that worked: **cash vs a cheque**. Cash is a shout: handed over, done, no record. A cheque is a slip: writing it moves no money, but because the instruction exists as a thing, you can post-date it, stop it, see it in your passbook, and present it again if it bounces.

**Command = write the instruction down instead of doing it right away.** You only bother when you need one of four powers:
**Later · Undo · Record · Retry**
If you need none of them, a normal method call is fine. Most code is shouts, and that's correct.
**Your own definition, which is exactly right:** "it is not performing the action, it is storing the action as an object."

| Restaurant | In code | Job |
|---|---|---|
| Slip | **Command** | knows **what**: holds the request |
| Chef | **Receiver** | knows **how**: does the real work |
| Rail | **Invoker** | knows **when**: holds slips and calls them, never reads them |

You had already used it: `executor.submit(() -> sendEmail(user))`. A `Runnable` is a command; the thread pool is the rail. (The word is **invoker**, not revoker: it *invokes*.)

### Shout or slip? (you scored 5 of 6)

| Situation | Answer |
|---|---|
| A button shows "Hello" | shout |
| Paint app with Ctrl+Z | slip — **undo** |
| "Send email tomorrow at 9 am" | slip — **later** |
| Chess app replays every move | slip — **record** |
| Bank transfer fails, retry in 5 minutes | slip — **retry** **(you changed to "shout")** |
| Calculator, tap `=` | shout |

Retry needs a slip because **you can't repeat something you didn't keep**. It's the bounced cheque: you can present it again only because you still have the paper.

## Undo

**Your first idea: "Ctrl+Z becomes a slip."** Flip it: **every action is a slip**, kept on a pile. Ctrl+Z just means "take the newest slip and ask it to reverse itself." That pile is a stack, the one from your DSA practice.

| Kind of undo | Example | The slip must remember |
|---|---|---|
| **Do the opposite** | Volume Up ↔ Volume Down | nothing |
| **Put back what was there** | Set to 30 → back to what it was | the old value |

**Your idea for "Set volume to 30": look back at the earlier slip that set it to 12.** There may be no such slip. The volume might have reached 12 through three "Volume Up" presses, or been 12 since switch-on. A slip can't rely on its neighbours; it carries its own way back.

```
class SetVolume implements Command {
    private final Tv tv;
    private final int newVolume;
    private int oldVolume;                    // filled in when execute() runs

    public void execute() {
        oldVolume = tv.getVolume();           // look first
        tv.setVolume(newVolume);              // then change
    }
    public void undo() { tv.setVolume(oldVolume); }
}

class Remote {                                // the invoker
    private final Deque<Command> history = new ArrayDeque<>();
    void press(Command c) { c.execute(); history.push(c); }
    void undo() { if (history.isEmpty()) return; history.pop().undo(); }
}
```

**Rules:** every slip must be able to undo itself, **alone**. If it overwrites something, it saves the old value **inside** `execute()`, just before overwriting, not when the slip is created. A slip made at 10 pm may run at 6 am.
Look at what's missing from `Remote`: the words "volume" and "Tv" never appear. It only knows `Command`.

## Exercise 26 — Ghar (smart home)

| Req | What it said | Answer | You |
|---|---|---|---|
| R1 | Buttons plus Undo, up to 10 steps | Command; invoker caps the history | ✓ |
| R2 | "Good night": lights, AC, door | Macro command | ✓ |
| R3 | Geyser at 6 am | Command, held by a scheduler | ✓ |
| R4 | **Whenever** the door is unlocked: notify, log | **Observer**, on the door | ✗ **put it in the command** |
| R5 | Fan speed 0 to 5 | Nothing, a check | ✓ |
| R6 | Voltas SDK is Fahrenheit only | Adapter | ½ "only one SDK, so no adapter" |

**R4:** "whenever X happens" is about the **thing**, not about one way of causing it. A door can be unlocked by the app, by undoing Good night, or by the keypad. Put the announcement where every route passes through: the door publishes, listeners react. **You even called it "a side effect" yourself.**
**R6:** Adapter isn't about how many vendors you have. It's about a **shape mismatch with code you can't change**. One vendor is enough. Otherwise the conversion is written in every caller.

**The fan bug hiding in R5.** Fan is at 5. "Speed up" does nothing, but the slip goes on the pile. Undo calls "speed down" and the fan drops to 4. **Undo reversed something that never happened.**
**"Put back what was there" is safer than "do the opposite."** Save the old speed in `execute()` and restore it. Also put the 0-to-5 limit inside `Fan`, so it protects itself.

### Ghar, Q2 to Q5

**Q2** ✓ AC at 22, set to 26, Undo: the slip saved 22 and puts it back. It learns the 22 inside `execute()`.
**Q3** ✓ Undo "Good night" in **reverse order**: unlock, restore AC, lights. One slip on the pile, so one tap undoes all three. Each inner slip restores **its own** past, so a light that was already off stays off.

```
class MacroCommand implements Command {
    private final List<Command> steps;
    public void execute() { for (Command c : steps) c.execute(); }                          // 1, 2, 3
    public void undo()    { for (int i = steps.size() - 1; i >= 0; i--) steps.get(i).undo(); }  // 3, 2, 1
}
```

**Q4** ✓ **You diagnosed it and guessed the fix: "store the commands in the database."** The queue lives in memory; a restart wipes it. A slip is three facts, and those are plain data:

| id | type | details | run_at | status |
|---|---|---|---|---|
| 81 | `TURN_ON_GEYSER` | `{"geyser":"bathroom-1"}` | 06:00 | PENDING |

A scheduler checks every minute for rows where `status = 'PENDING' AND run_at <= now()`, rebuilds each slip with a factory, executes it, and marks it DONE. `<=`, not `=`, so a missed minute is caught on the next check.

**Q5** ✓ **"Water the plants" can't be undone.** You considered throwing and rejected it because of **Liskov**, correctly: the invoker trusts every `undo()` to work. Better than logging silently:
**1. Don't put it on the pile.** Split the interface, which is Interface Segregation used properly:

```
interface Command { void execute(); }
interface UndoableCommand extends Command { void undo(); }
```

**2. Tell the user.** Grey out Undo, or confirm before running anything irreversible.
**3. Do the nearest useful thing.** You can't un-water a plant, but you can **stop** the valve. You can't un-pay, but you can refund. That's *compensation*.

### The bug hunt (you found 3 of 5)

| # | Bug | Found? |
|---|---|---|
| 1 | `undo()` sets `newTemp` instead of `oldTemp` | ✓ |
| 2 | Macro `undo()` loops forwards | ✓ |
| 3 | `history.peek().undo()` never removes the slip | ✓ |
| 4 | `oldTemp` read in the **constructor**, not in `execute()` | **missed** |
| 5 | `history.push(c)` **before** `c.execute()` | **missed** |

**Clue for #4:** the field was `final`. A final field can only be set in the constructor, and the constructor is the wrong moment. **#5:** if `execute()` throws, a slip that never happened sits on the pile.

> **Two rules:** save the old state at the last possible moment, and **only record what actually happened** (execute first, then push).

### Your Part B: the text editor — it worked

**Your first Part B that produced the correct output.** Both slips saved the old text inside `execute()`, the field wasn't final, every method was public, the empty check was there, and the invoker knew only `Command`.

> **Two small things:** the imports for `Deque` and `ArrayDeque` were missing (the only compile error); and `press()` pushed before executing, which is bug #5 from the hunt.
> **Naming:** `currentText` holds the text from *before* the change, so `previousText`.
> **Trade-off to name if asked:** you snapshot the whole document per slip. Safe, but a 5 MB document × 100 actions is 500 MB of history. The alternative is to remember only what was added.

### Part C

> **Q1 — Redo was new to you.** Redo means "undo my undo." Don't throw an undone slip away; move it to a second pile.

```
                     history        redo pile      document
after A, B, C        [A, B, C]      [ ]            Hello Pune!
Undo                 [A, B]         [C]            Hello Pune
Undo                 [A]            [C, B]         Hello
Redo                 [A, B]         [C]            Hello Pune
(new action D after two undos)
                     [A, D]         [ ]            Hello Mumbai      redo pile is CLEARED
```

```
void press(Command c) { c.execute(); history.push(c); redo.clear(); }
void undo() { if (history.isEmpty()) return; Command c = history.pop(); c.undo();    redo.push(c); }
void redo() { if (redo.isEmpty())    return; Command c = redo.pop();    c.execute(); history.push(c); }
```

A new action empties the redo pile because those slips belonged to a future that no longer exists. Not one command class changes: redo lives entirely in the invoker.

> **Q2 — "Good night" fails at the lock. ✓ You chose all-or-nothing.** Rolling back *does* need a catch: catch, undo what ran (in reverse), then rethrow.

| Catch, then… | Meaning | Where |
|---|---|---|
| carry on | this failure doesn't matter | Compress failing |
| throw your own exception | translate the error | the MapMyRide adapter |
| **clean up, then rethrow** | undo what I did, still report failure | the macro command |

**But is rollback what the user wants?** At 11 pm it turns the lights back on and the door is still unlocked. The test: **do the steps only make sense together?** A bank transfer: yes, roll back. Independent steps: keep what worked and alert loudly. **Never report a partial failure as success.**

> **Q3 — "Java has lambdas. Why classes?" ✓ Your best defence answer, with the concession first.** You said: a lambda only fits an interface with one method, and Command has two. Correct. One fix: a lambda *can* carry variables in; what it can't do is remember something **between** two calls.
> A class is needed for: **undo** (two methods) · **memory between execute and undo** (a field) · **saving to a database** (a type name and parameters) · **a name** for logs and menus.
> **Script:** "You'd be right for run-only actions, and I'd use a lambda there. I'd write a class once the command needs undo, needs to remember state, or has to be saved and rebuilt after a restart."

### Bill Pugh vs "create once and pass it in"

| | Bill Pugh | Pass it in |
|---|---|---|
| Can you see what the class depends on? | No, hidden inside a method | Yes, in the constructor |
| Can a test use a fake? | Hard | Easy |
| Could you ever have two? | No | Yes |

> **Say it:** "I need one shared instance. I'd create it once and inject it. If we can't inject, I'd use a Bill Pugh singleton." Bill Pugh is still the best answer when asked to *write a thread-safe singleton*.

# 18. Jumbled round 2 — PuneGo

A ticket app for Pune's metro, buses and e-rickshaws. Nine patterns in play, no hints, three requirements that need none.

## Q1 — you scored 10½ out of 11

| Req | What the PM said | Answer | You |
|---|---|---|---|
| R1 | Fares differ per mode; mode arrives as text; a new mode each year | Strategy + Factory | ✓ |
| R2 | On booking: SMS, points, dashboard. Dashboard often down | Observer | ✓ |
| R3 | A ticket is always the same five fields | Nothing, a record | ✓ |
| R4 | PayZap's `charge(paise, ref)` vs our `pay(rupees)` | Adapter | ✓ |
| R5 | Student, senior, peak, festival; several can apply | Decorator, around the strategy | ✓ |
| R6 | BOOKED → ACTIVE → USED, or CANCELLED, or EXPIRED, with rules | State | ✓ |
| R7 | No payment, no ticket | Nothing: a required step, called directly | ✓ |
| R8 | Trip search: 2 required, many optional, a cross-field rule, immutable | Builder | ✓ |
| R9 | Crowd count must be the same on every screen | One shared instance | ✓ |
| R10 | Book now, book every weekday, Undo within 2 minutes | Command | ✓ |
| R11 | Unscanned for 2 hours: expires on its own | Nothing: a scheduled job | ½ |

On the parking lot, the same kind of question got 6½ out of 10. Every costume that fooled you before, you saw through here.

**R11 — half the answer, and a good half.** You said: check the ticket's age when it is scanned. That works for a ticket that gets scanned. But a rider who never shows up is never scanned, so the ticket stays BOOKED forever. "Hasn't happened in N hours" means nothing triggers it, so a **scheduled job** must go looking, and call `ticket.expire()` through the state machine.
**The best design uses both:** the job for cleanup, your scan-time check as the guard at the gate for the minutes between runs. **And you did not say Observer, which was the trap in the State exercise.**

**Five small sharpenings**
**R2:** the try/catch goes **once, in the loop** that calls the listeners, not inside each listener.
**R4:** the reason for an adapter isn't "a new SDK might come." It's "their shape doesn't match ours and we can't edit theirs."
**R6:** five statuses, so five classes, not four.
**R8:** you described a Builder perfectly and never said the word. Say the name.
**R9:** Bill Pugh again. One instance, created once and injected.

## Q2 — the wiring for a student at peak hour

**You wrote:** `new Student(new Peak(new PricingFactory("Metro").createObj().calculatePrice()))`
The `.calculatePrice()` at the end turns the strategy into a **number**. A decorator wraps an **object**, so it can ask for the price later. It's like writing `new Adrak(20)` instead of `new Adrak(new MasalaChai())`.

```
FareStrategy fare = new StudentDiscount(new PeakSurcharge(factory.forMode("METRO")));
int price = fare.fareFor(trip);        // one call, on the outside; it travels inward by itself
```

✓ **Your Finance question was the right one.** With a metro fare of 30 and a peak charge of 5:

```
Student( Peak( Metro ) )     30 + 5 = 35  → half  = 17.50
Peak( Student( Metro ) )     half = 15    → + 5   = 20
```

A second question worth asking: on a festival day, is the ride free even at peak hour?

## Q3 — the steps inside `bookTicket()`

✓ **You had the key order: pay, then mark booked, then announce.** Three adjustments: the ticket is the **result** (it holds the fare, so it can't exist before pricing and payment); **save** it before announcing; and if saving fails after payment, **refund**.

```
Ticket bookTicket(BookingRequest request) {
    1. validate                from ≠ to, mode exists
    2. price                   strategy (R1) wrapped in decorators (R5)
    3. pay                     through the adapter (R4). REQUIRED: if it throws, stop. (R7)
    4. create the ticket       starts life as BOOKED (R3, R6)
       and save it             if saving fails → REFUND, then throw
    5. announce                SMS, points, dashboard (R2), each in try/catch
    6. return the ticket
}
```

**The shape to remember:** validate → price → required steps → record → announce. Each step runs only if every earlier one worked.

## Q4 — three servers ✓

You said: move the tracker to Redis. Correct. The thing to add is what goes wrong even when all servers are healthy: three servers means **three separate maps**, each counting only its own bookings, so two screens show different numbers. **A Singleton is one per JVM, not one per system.** Redis `INCR` is atomic across servers, the way `AtomicInteger` is inside one JVM.

```
interface CrowdTracker { void increment(String trainId); int count(String trainId); }
class InMemoryCrowdTracker implements CrowdTracker { ... }
class RedisCrowdTracker    implements CrowdTracker { ... }     // one new class, one wiring line
```

## Q5 — the rider taps Undo a minute after booking

You saw that the money, the status and the count must all be reversed. The two patterns:

**Command** gives the Undo button. **State** decides whether Undo is allowed.

```
class BookTicket implements Command {
    private Ticket ticket;
    public void execute() { ticket = bookingService.bookTicket(request); }
    public void undo()    { bookingService.cancel(ticket); }
}

void cancel(Ticket ticket) {
    ticket.cancel();                          // 1. state machine: allowed only from BOOKED
    gateway.refund(ticket.paymentRef());      // 2. the money
    publish(new TicketCancelled(ticket));     // 3. tell everyone
}
```

If the rider walked through the gate before tapping Undo, the ticket is ACTIVE, `cancel()` throws, and no refund happens. The command needed no `if` for that.

| What happened | Literally undoable? | So you… |
|---|---|---|
| Ticket became BOOKED | no deleting | move it to **CANCELLED** |
| Money was charged | you can't un-pay | **refund** |
| SMS was sent | you can't un-send | send a **new** "cancelled" SMS |
| Crowd count, loyalty points | yes | decrement, remove |

> **Say it:** "Undo is a command. Its undo calls cancel on the ticket, and the state machine decides if that's allowed. If it is, I refund through the gateway and publish a cancelled event so the listeners reverse their side effects. The ticket isn't deleted; it moves to CANCELLED."

# 19. The hands-on build — PuneGo in four stages

You asked for hands-on practice at weaving patterns together. Each stage added one pattern on top of the last; a small `main` tested each piece before the final assembly.

| Stage | You wrote | Pattern | Result when I ran it |
|---|---|---|---|
| 1 | three fares, a registry factory | Strategy + Factory | correct after adding two imports; message read `Unknown modeCAB` |
| 2 | `StudentDiscount`, `PeakSurcharge` | Decorator | **ran clean, exact output** |
| 3 | `PayZapAdapter` | Adapter | **ran clean; all four translations, including `100L`** |
| 4 | `BookingService` | Observer + weaving | ran clean; two values differed (fare 30 vs 25, id `1` vs `T1`) |

**Stage 4 — Asha paid 30 instead of 25.** Finance said: peak first, then the discount on the total. You wrapped student first, then peak, which builds `Peak( Student( Metro ) )` = 20 + 10 = 30.

**The layer you wrap first is innermost, and the innermost runs first.** So "peak first" means *wrap peak first*. Nothing crashes when this is wrong; only checking the output against the expected number catches it.

### The finished service

```
class BookingService {
    private final FareFactory factory;
    private final PaymentGateway gateway;          // the interface; it doesn't know an adapter is behind it
    private final List<BookingListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger nextId = new AtomicInteger(1);

    BookingService(FareFactory factory, PaymentGateway gateway) {
        this.factory = factory;
        this.gateway = gateway;
    }

    public void subscribe(BookingListener listener) { listeners.add(listener); }

    Ticket bookTicket(BookingRequest request) {
        FareStrategy strategy = factory.forMode(request.trip().mode());      // Factory → Strategy
        if (request.peak())    strategy = new PeakSurcharge(strategy);       // Decorator: inner, runs first
        if (request.student()) strategy = new StudentDiscount(strategy);     // Decorator: outer, on the total
        int fare = strategy.fareFor(request.trip());

        gateway.pay(fare);                                                    // Adapter; REQUIRED, may throw

        Ticket ticket = new Ticket("T" + nextId.getAndIncrement(), request.rider(), fare);

        for (BookingListener listener : listeners) {                          // Observer, announced LAST
            try {
                listener.onBooked(ticket);
            } catch (RuntimeException e) {
                System.out.println("Listener failed: " + e.getMessage());
            }
        }
        return ticket;
    }
}
```

```
PayZap charged 2500 paise for PUNEGO-01
SMS to Asha: ticket T1 booked, fare 25
Listener failed: dashboard is down
Loyalty points added for Asha
Got ticket T1 for 25
Unknown mode: CAB                                ← Ravi: no SMS, no points
Payment of 2000 rupees failed: limit exceeded    ← Meera: no SMS, no points
```

**Read the output like a story.** Asha gets everything. The dashboard crashes and loyalty still runs. Ravi and Meera get nothing announced, because their bookings failed before a ticket existed. That is "required steps first, announce last" made visible.

**Why a new `main` each stage?** You asked this. `main` isn't part of the app; it's a small test for the piece being built. The classes all stay. It's like checking the wheel spins before fitting it to the bike.

# 20. Consolidation

## Requirement → pattern cheat sheet (all nine)

| What the requirement sounds like | Answer |
|---|---|
| "a different formula per type", "we change our minds twice a year" | **Strategy** |
| "the type arrives as text and we must pick the right object" | **Factory** |
| "must be the same number everywhere", "one source of truth" | **one shared instance**, injected |
| "2 required, many optional, a rule across fields, never changes after" | **Builder** |
| "when X happens also do A, B, C" and "X still works if one fails" | **Observer** |
| "allowed in some statuses, refused in others", many statuses × many operations | **State** |
| "several extras can apply together, in combinations" | **Decorator** |
| "their SDK doesn't match our interface, and we can't edit it" | **Adapter** |
| "do it later", "undo", "replay", "retry", "every weekday" | **Command** |
| "no A, no X" (payment, barrier, access card) | **nothing**: a required step, called directly |
| "X hasn't happened in N hours" | **nothing**: a scheduled job |
| "always the same few fields" | **nothing**: a record |
| "applies to every single one, always" | **nothing**: put it in the base |
| "we wrote it ourselves, it just needs a better name" | **nothing**: rename it |
| "maybe next year" | **nothing**: YAGNI |

## New rules from this group

| Area | Rule |
|---|---|
| Decorator | Same interface as the thing it wraps, adds a bit on top. |
| Decorator | Order matters. The layer you wrap first is innermost and runs first. |
| Decorator | Wrap **objects**, not numbers. Ask for the result once, from the outermost layer. |
| Decorator | Every layer must use what the inner layer returns. |
| Decorator | If it applies to everything, always, put it in the base. |
| Decorator | Changes the thing passing through → Decorator. Reacts afterwards → Observer. |
| Pipelines | Compress, then encrypt. A layer that reads content runs while it is still readable. |
| Pipelines | Build the stack in one factory so only valid orders exist. |
| Adapter | Keeps the behaviour, changes the shape. Translates names, types, order, units, errors. |
| Adapter | Keep adapters thin. Extra behaviour goes in a decorator around it. |
| Adapter | It's for code you can't change. If you own it, change it. |
| Adapter | Same-type values can be swapped silently (lat/lng). Check order and units. |
| Command | The action stored as an object. Needed for: later, undo, record, retry. |
| Command | Slip knows what, receiver knows how, invoker knows when. |
| Command | Each slip undoes itself alone. Save the old value inside `execute()`. |
| Command | "Put back what was there" is safer than "do the opposite." |
| Command | Execute first, then record. Only record what actually happened. |
| Command | Macro undo runs in reverse. Redo is a second pile, cleared by any new action. |
| Command | A command is data, so it can be saved in a table and survive a restart. |
| Failure | try/catch is for continuing. To stop, let it throw. |
| Failure | Three uses of catch: carry on · translate · clean up then rethrow. |
| Failure | What can't be rewound gets compensated: refund, a "cancelled" SMS, stop the valve. |
| Failure | Never report a partial failure as success. |
| Design | "Whenever X happens" belongs to the thing, not to one way of causing it. |
| Design | Many boolean flags in a method call → pass one settings object. |
| Design | One instance, created once and injected. Bill Pugh only when you can't inject. |
| Interview | On "over-engineered": start with "You'd be right if…", then defend. |

## Mistake tracker

| Mistake | Status after this group |
|---|---|
| Missing `public` on interface implementations | **gone since Decorator** |
| Calling a method with no receiver | **gone since Decorator** |
| Wrong or missing return types | **not seen in this group** |
| Missing imports ( `Map` , `Deque` , `Scanner` ) | **three times; fixed by stage 2 of the build** |
| Several `public class` in one file | **Katta; fixed afterwards** |
| Skipped requirement | **still appearing: all layers on every message, retry "thrice", id `1` not `T1`** |
| Decided but not coded | **still appearing, but smaller: wrap order in stage 4; "stop" implemented as try/catch** |
| Discarding the inner result in a decorator | Katta footer; correct in the build |
| Recording before executing ( `push` then `execute` ) | **repeated right after the bug hunt** |
| Extra behaviour inside an adapter or a command | Dabba R3, Ghar R4; correct in PuneGo |
| Reaching for Bill Pugh | **three times** |
| Naming principles that don't apply | Adapter Q2; used correctly (Liskov) in Ghar Q5 |
| Not conceding on "defend it" | **conceded unprompted on Command Q3** |
| Not running the code | by choice, for time. I run it for you; expected output is given to check against |

**Where you stand.** **Recognition:** 6½ / 10 on the parking lot → 10½ / 11 on PuneGo. Interview-ready.
**Code:** about thirty compile errors in the State exercise → three stages in a row that ran with no fixes.
**Habit that appeared:** catching your own mistake mid-answer (Katta R5 and Q4, Dabba Q4, Ghar R4's "side effect").
**The remaining gap:** small details between the decision and the code. The cure that worked this week: **check your output against the expected output, line by line.**

## What changed in how we work

| |
|---|
| Every pattern starts with a story and a few guessing questions, before any definition. |
| Extra Q&A rounds whenever a pattern hasn't clicked (Command needed two, and they worked). |
| Part B is small, with given starter code and expected output; I run it. |
| Bug hunts as a quick alternative to writing. |
| After a jumbled round, a staged build that weaves the patterns into one service. |

## Recall questions — cover the answers

1. 2 chai bases and 5 add-ons: how many classes with inheritance, and how many with decorators?
2. What one line makes a decorator wrappable again and again?
3. Strategy vs Decorator, in one sentence.
4. Member 10% off, festival +20, weekend ×1.5, base 80. Give both totals for discount-outermost and discount-innermost.
5. Which runs first: the layer you wrapped first, or last?
6. Why compress before encrypt? What were the three byte counts in the experiment?
7. Why must the profanity filter run before translation, compression and encryption?
8. "Every message gets a timestamp." Why is that not a decorator?
9. The footer bug: what one wrong variable discards every inner layer?
10. Encrypt fails: send or stop? Compress fails: send or stop? What is the test?
11. To stop a message when a layer fails, do you add a try/catch?
12. Decorator vs Adapter: what goes in and what comes out of each?
13. Name the five kinds of thing an adapter translates.
14. Why is `rupees * 100L` written with an `L`?
15. The SDK returns `{longitude, latitude}`. Why doesn't the compiler catch a swap?
16. Where do logging and retry go when the SMS vendor may change?
17. Your own team's class has a badly named method. Adapter?
18. A vendor's checked exception: swallow, let it escape, or something else?
19. What are the four powers that justify a Command?
20. Slip, receiver, invoker: which knows what, how and when?
21. Why does retry need a command object?
22. "Set volume to 30": what does the slip save, and on which line?
23. Fan at speed 5, "speed up", then Undo. What goes wrong, and what is the cure?
24. Why push to history only after `execute()`?
25. Undo of a macro command: which order, and why?
26. Redo: what is the second pile, and what empties it?
27. The server restarts at 2 am. How does the 6 am geyser command survive?
28. "Water the plants" then Undo: give three better options than a silent log.
29. "Good night" fails at step 3. When do you roll back, and when do you keep what worked?
30. When is a lambda enough for a command, and when do you need a class?
31. "Whenever the door is unlocked, notify." Why not inside the unlock command?
32. Write the six steps of `bookTicket()` in order.
33. Tickets unscanned for 2 hours must expire. Why isn't a check at scan time enough?
34. Three servers, one crowd count. What breaks and what fixes it?
35. Undo a booking: which two patterns cooperate, and what happens to the money and the SMS?
36. Bill Pugh or inject? Give the one-sentence interview answer.